Aethelgard: Factor-Graph-Driven Decision Making in an Interactive Narrative Adventure Game

CSE440.1 | Group 9
Soomaiya Chowdhury (2211856042), Mahmudur Rahman Labib (2131214642),
Shreysee Moni Shashy (2122043642), Shafi Mahadi Nowrose (2121620643)

***Abstract*—Interactive narrative games have long relied on scripted decision trees, where player choices lead to predetermined outcomes with little or no variation. While this approach has been commercially successful, it suffers from fundamental scalability and replayability limitations. A narrative with N binary decision points requires the authoring of 2^N distinct story paths, making deep branching computationally intractable for human writers. Furthermore, scripted branches offer no variation—choosing the same option always yields the same outcome, which diminishes replay value and breaks the illusion of a living, reactive story world.**

This paper presents *Aethelgard*, an interactive adventure quest game that replaces scripted branching with probabilistic inference powered by factor graphs. At each decision point, the system constructs a factor graph consisting of observed player statistics (health, resolve, morality) and hidden world variables (NPC trustworthiness, environmental threat, resource scarcity, knowledge value, and moral significance). Loopy belief propagation with Monte Carlo importance sampling is then performed to infer posterior beliefs about the hidden state of the game world. These inferred beliefs directly shape probabilistic outcomes including stat changes, NPC betrayals, and ambush events, replacing hardcoded deterministic rules with a mathematically grounded uncertainty model. A temporal carry-forward mechanism persists posteriors across scenes as informed priors, ensuring that the narrative remains consistent and that early decisions have lasting consequences throughout the story.

The game features a branching narrative with 20 story nodes organized across 5 stages, leading to 4 distinct endings. Each ending reflects a different thematic resolution: the morally compromised tyrant, the self-sacrificing guardian, the martyred hero, or the hollow survivor who preserved only themselves. The system is implemented as a web application with a Python Flask backend and a vanilla JavaScript frontend, including a real-time "AI Brain" visualization panel that displays the factor graph's inference state to the player, providing transparency into the probabilistic reasoning that drives the game world. Experimental playthroughs across multiple random seeds demonstrate that the probabilistic inference engine converges reliably within 3–5 iterations, produces progressively escalating threat and scarcity beliefs consistent with narrative tension, and generates a roughly uniform distribution of endings—confirming that outcomes are genuinely driven by player decisions rather than structural biases in the model.

***Index Terms*—Factor graphs, belief propagation, interactive narrative, probabilistic graphical models, game artificial intelligence, Monte Carlo methods**

I. Introduction

The intersection of artificial intelligence and interactive storytelling has been an active area of research since the earliest days of computational narrative [1]. The fundamental challenge is to create story worlds that respond meaningfully to player agency while maintaining narrative coherence—a task that requires balancing authorial intent with emergent, player-driven outcomes. In commercial game development, the dominant approach remains the scripted decision tree, where human authors predefine every possible story path and the consequences of each choice. While this method gives writers precise control over the narrative experience, it imposes severe constraints on the depth and reactivity of the story.

The scalability problem of scripted branching is well-documented. A binary narrative with 10 decision points already requires 1,024 distinct story paths, and a 20-point narrative would demand over one million. Even with compression techniques such as merging common story beats or using directed acyclic graphs instead of trees, the authoring burden remains prohibitive for deep narratives. More fundamentally, scripted outcomes are deterministic: the same choice always produces the same result, regardless of the context that preceded it. This rigidity undermines the player's sense of agency and eliminates replayability, since a player who has completed one path has effectively experienced all paths.

Probabilistic graphical models offer a principled solution to these limitations [2], [3]. Among these models, factor graphs are particularly well-suited to the interactive narrative domain because they provide a clear, modular representation of the relationships between observable quantities (player actions and statistics) and latent quantities (the hidden state of the story world). A factor graph is a bipartite graph consisting of variable nodes—which represent random variables—and factor nodes—which encode compatibility functions over subsets of those variables. By performing inference on the factor graph through iterative message passing (belief propagation), the system can reason about hidden aspects of the game world in a mathematically grounded way, producing outcomes that are probabilistic rather than deterministic, and context-dependent rather than scripted.

In this paper, we present *Aethelgard*, an interactive adventure quest game that uses factor graphs as the core decision-making engine for all narrative outcome generation. The player assumes the role of an Investigator navigating the ruins of a collapsed medieval kingdom, making choices at each of 5 narrative stages that affect both visible statistics (health points, mental resolve, moral alignment) and hidden world variables (NPC trustworthiness, environmental danger, resource scarcity, knowledge value, and moral significance). At every decision point, the engine constructs a fresh factor graph, performs

loopy belief propagation to infer posterior beliefs about the hidden variables, and uses those beliefs to probabilistically determine the consequences of the player's choices—including stat changes of varying magnitude, NPC betrayals triggered by low trust beliefs, and ambush events triggered by high threat beliefs.

A critical design feature is the temporal carry-forward mechanism: after inference completes at each decision point, the posterior distributions of all hidden variables are snapshotted and used as informed priors for the next scene's factor graph. This creates a temporally consistent model of the player's journey, where early decisions that affect trust or threat levels propagate their influence throughout the entire narrative, leading to outcomes that feel consequential and coherent rather than isolated.

The main contributions of this work are:

1) A factor graph model with 9 variable nodes (3 observed, 6 hidden) and 7 factor nodes that captures the relevant dynamics of an adventure narrative, including threat amplification, NPC betrayal, moral alignment, resource cost, and survival utility.
2) A temporal carry-forward mechanism that preserves belief consistency across scenes, enabling compound effects from sequential decisions.
3) A Monte Carlo importance sampling procedure within factor nodes that approximates intractable marginals through weighted sampling, with a blending strategy that prevents overconfident beliefs.
4) A web-based game engine with a real-time factor graph visualization panel ("AI Brain") that provides players with transparent insight into the probabilistic reasoning driving their outcomes.
5) A branching narrative with 20 story nodes and 4 endings where outcomes emerge from inference rather than scripted rules.

The remainder of this paper is organized as follows. Section II reviews the theoretical background on factor graphs and belief propagation, and discusses related work in interactive narrative and game AI. Section III describes the factor graph model in detail, including graph construction, factor definitions, the inference procedure, and the temporal consistency mechanism. Section IV details the system architecture across all three tiers. Section V presents the narrative design, including story structure, character system, and player statistics. Section VI reports experimental results on convergence, belief evolution, and ending distribution. Section VII concludes the paper and discusses directions for future work.

To provide concrete context for the technical contributions described in this paper, it is worth briefly summarizing the player experience. Upon launching the game, the player is presented with a prologue screen that introduces the setting: the year is 2026, and the kingdom of Aethelgard has fallen silent behind locked iron gates after a mysterious psychological madness consumed its population. The player, as an Investigator, must navigate the ruins, face the moral weights of those left behind, and uncover the truth. Each subsequent scene presents dialogue from characters encountered in the ruins, followed by a binary choice that affects the player's visible statistics and triggers factor graph inference on the hidden world variables. An optional "AI Brain" panel can be toggled to visualize the inference process in real time, showing how the factor graph's beliefs evolve in response to each decision. The experience culminates in one of four endings, each reflecting the cumulative consequences of the player's choices throughout the journey.

II. Background and Related Work

A. Factor Graphs

A factor graph is a type of probabilistic graphical model that explicitly represents the factorization of a joint probability distribution over a set of random variables [2]. Formally, a factor graph $G = (V, F, E)$ consists of a set of variable nodes $V = \{x_1, \ldots, x_n\}$, a set of factor nodes $F = \{f_1, \ldots, f_m\}$, and a set of edges E that connect factor nodes to the variable nodes they depend on. The joint distribution factorizes as:

$$p(x_1, \ldots, x_n) = \frac{1}{Z} \prod_{j=1}^{m} f_j(x_{N_j}) \tag{1}$$

where N_j denotes the set of variables connected to factor f_j, and Z is a normalization constant.

Factor graphs are bipartite by construction: edges only connect factor nodes to variable nodes, never factor-to-factor or variable-to-variable. This structure enables efficient inference algorithms, most notably the sum-product algorithm (belief propagation), which computes marginal distributions by passing messages along the edges of the graph [2]. On tree-structured graphs, the algorithm produces exact marginals. On graphs with cycles—as encountered in many practical applications—iterative loopy belief propagation provides approximate but often useful results [3].

Each variable node in a factor graph maintains a belief, which is a probability distribution representing the current estimate of that variable's value. In our implementation, beliefs are parameterized as Gaussian distributions $\mathcal{N}(\mu, \sigma^2)$, where μ is the mean and σ^2 is the variance. This Gaussian parameterization enables efficient belief updates through precision-weighted averaging, a technique rooted in Bayesian estimation theory [3].

B. Belief Propagation

Belief propagation, also known as the sum-product algorithm, is an iterative message-passing algorithm for computing marginal distributions in graphical models [2], [3]. The algorithm proceeds in two alternating phases.

Factor-to-variable messages: Each factor node f_j computes an outgoing message for each connected variable x_i by marginalizing out all other variables:

$$\mu_{f_j \to x_i}(x_i) = \sum_{x_{N_j \setminus \{x_i\}}} f_j(x_{N_j}) \prod_{x_k \in N_j \setminus \{x_i\}} \mu_{x_k \to f_j}(x_k) \tag{2}$$

Variable-to-factor messages (belief updates): Each variable node combines all incoming messages to form its updated belief:

$$b(x_i) \propto \mu_{\text{prior}}(x_i) \prod_{j} \mu_{f_j \to x_i}(x_i) \tag{3}$$

On tree-structured factor graphs, this algorithm converges to exact marginals in a number of steps equal to the graph diameter. On loopy graphs, the algorithm is applied iteratively until a convergence criterion is met or a maximum iteration count is reached. While loopy belief propagation does not guarantee exact inference, it has been shown to produce good approximations in a wide range of applications, including computer vision [4], error-correcting codes [5], speech recognition, and gene regulatory network inference.

A key practical consideration is the choice of convergence criterion. In our implementation, we monitor the maximum change in any variable's posterior mean between successive iterations and terminate when this change falls below a tolerance threshold ε. This provides a simple and effective stopping condition that balances accuracy against computational cost.

C. Interactive Narratives and Game AI

The use of artificial intelligence for interactive storytelling dates back to the PLATO system in the 1960s and 1970s [6], which featured text-based adventure games with branching narratives. Since then, the field has evolved through several generations of approaches.

Early story generation systems focused on grammatical and structural models of narrative, applying formal language theory to generate story sentences [1]. Drama management systems introduced the concept of an AI director that monitors player engagement and adjusts narrative events accordingly. More recent work on narrative planning [7] uses AI planning algorithms to generate story arcs that satisfy both plot constraints and character goals, producing narratives that are coherent and dramatically satisfying.

In the commercial game industry, the dominant approach to interactive narrative remains the hand-crafted decision tree, as seen in Telltale's *The Walking Dead* and Quantic Dream's *Detroit: Become Human*. These games offer the illusion of meaningful choice through well-written branching paths, but the underlying structure is entirely scripted and deterministic. Factor graphs have been applied in game AI primarily for player modeling—inferring player preferences, skill levels, and emotional states from observed behavior [8]. Their application as the core decision engine for narrative outcome generation, as in our work, is relatively unexplored. Yannakakis and Togelius [8] discuss several directions for AI in games, including procedural content generation and player modeling, but do not specifically address factor-graph-driven narrative inference.

Our work differs from these existing approaches in a fundamental way: rather than scripting outcomes or using the factor graph for auxiliary tasks such as player modeling, we use the factor graph as the primary mechanism for determining what happens in the story. Every outcome—stat changes, betrayals, ambushes—is generated by sampling from distributions shaped by the graph's inferred beliefs. This makes the narrative genuinely emergent rather than preauthored.

The use of Bayesian inference in games has also been explored for difficulty adaptation and player modeling. Our contribution extends this line of work by using Bayesian inference not for player modeling but for world-state inference—reasoning about hidden aspects of the game world rather than hidden aspects of the player.

From a software engineering perspective, our three-tier architecture (game logic, web server, client presentation) follows established patterns for web-based games. The separation of narrative content (stored in JSON) from game logic (implemented in Python) enables easy expansion of the story without modifying code, a principle borrowed from data-driven game design.

III. Factor Graph Model

A. Graph Structure

At each decision point in the narrative, the engine constructs a factor graph consisting of 9 variable nodes and 7 factor nodes. The variable nodes are divided into two categories based on whether they represent known evidence or latent quantities that must be inferred.

Observed variables represent player statistics that are directly known and cannot change during inference. Their beliefs are clamped (fixed) throughout the message-passing process, and incoming messages are discarded rather than updating their values. This reflects the fact that the player's current HP, resolve, and morality are measurable, observable facts about the game state.

Hidden variables represent latent aspects of the game world that are not directly observable but can be inferred from the relationships encoded in the factor structure. These include the current environmental threat, the trustworthiness of NPCs, the scarcity of resources, the value of available knowledge, and the moral weight of the pending decision.

TABLE I
Variable Nodes

Variable	Type	Description	Prior
hp	Observed	Player health points	$\mathcal{N}(\text{hp}, 2)$
resolve	Observed	Player mental resolve	$\mathcal{N}(\text{res}, 2)$
morality_stat	Observed	Player morality score	$\mathcal{N}(\text{mor}, 2)$
threat_level	Hidden	Environmental danger	$\mathcal{N}(\mu_{\text{threat}}, 18)$
npc1_trust	Hidden	Trustworthiness of NPC1	$\mathcal{N}(\mu_{\text{npc1}}, 25)$
npc2_trust	Hidden	Trustworthiness of NPC2	$\mathcal{N}(\mu_{\text{npc2}}, 25)$
resource_scarcity	Hidden	Scarcity of resources	$\mathcal{N}(\mu_{\text{scarc}}, 15)$
knowledge_value	Hidden	Value of information	$\mathcal{N}(\mu_{\text{know}}, 20)$
moral_weight	Hidden	Moral significance	$\mathcal{N}(\mu_{\text{moral}}, 20)$

The prior means for hidden variables are computed from heuristic functions of the current game state. The threat level mean, for example, is initialized as:

$$\mu_{\text{threat}} = \min(\text{base\_threat} + \text{fatigue} \times 0.3,\ 95) \qquad (4)$$

where $\text{base\_threat} = 40 + n_{\text{choices}} \times 5$, $\text{fatigue} = \min(n_{\text{choices}} \times 12,\ 100)$, and n_{choices} is the total number of decisions the player has made so far. This ensures that the game world becomes progressively more dangerous as the narrative advances. The initial variance of each hidden variable controls how uncertain the system is at the start of each scene—higher variance allows more room for inference to shift beliefs, while lower variance constrains them.

B. Factor Definitions

The 7 factor nodes encode compatibility relationships between variables. Each factor function takes a dictionary of sampled variable values and returns a scalar score in $[0, 100]$ indicating how compatible those values are under the modeled relationship. Higher scores indicate greater compatibility.

TABLE II
FACTOR NODES

Factor	Variables	Function
f_{danger}	threat, hp	thr×(1 + max(0, 100−hp)/200)
f_{bet1}	npc1, threat	(100−tr)×0.4 + thr×0.3
f_{bet2}	npc2, threat	(100−tr)×0.35 + thr×0.25
f_{moral}	moral, t1, t2	$100 - \lvert\text{mor} - \text{avg}(t1, t2)\rvert$
f_{cross}	t1, t2	$100 - \lvert t1 - t2\rvert \times 0.8$
f_{cost}	scarc, mor, res	scr×0.4+mor×0.3+(100−res)×0.2
f_{util}	know, thr, mor	(100−thr)×0.4+knw×0.3+(100−mor)×0.15

The f_{danger} factor models the compounding effect of low health on perceived threat: when HP is low, even moderate threat levels feel more dangerous. The f_{bet} factors encode the intuition that NPCs are more likely to betray the player when trust is low and the overall threat level is high (creating pressure for self-serving behavior). The two betrayal factors use slightly different weights (0.4 vs. 0.35 for trust, 0.3 vs. 0.25 for threat) to create asymmetric trust dynamics between the two NPCs.

The f_{moral} factor captures the alignment between the player's moral stance and the average trustworthiness of the two NPCs: when morality closely matches average trust, the score is high, indicating compatibility. The f_{cross} factor penalizes extreme disparities between the two NPC trust values, encouraging the model to maintain balanced trust unless evidence strongly favors one NPC over the other.

The f_{cost} factor determines how expensive a choice is in terms of stat losses, with resource scarcity having the largest weight (0.4), followed by moral weight (0.3) and low resolve (0.2). The f_{util} factor measures the survival benefit of a choice, weighting low threat (0.4), high knowledge (0.3), and low moral burden (0.15).

All factor outputs are clamped to the range $[0, 100]$ to maintain consistent scaling and prevent numerical issues.

C. Inference Procedure

Inference is performed using loopy belief propagation, which alternates between factor-to-variable message computation and variable belief updates until convergence.

Phase 1: Factor-to-Variable Messages via Monte Carlo Importance Sampling. Since the factor functions are nonlinear, exact marginalization is analytically intractable. We instead approximate the outgoing message for each factor using Monte Carlo importance sampling with $N = 300$ samples. For a factor f connected to variables $\{v_1, \ldots, v_k\}$, the message to variable v_i is computed as follows:

1) Draw 300 samples from the current beliefs of all connected variables: $v_j^{(s)} \sim \mathcal{N}(\mu_j, \sigma_j^2)$ for $j = 1, \ldots, k$ and $s = 1, \ldots, 300$. Each sample is clamped to $[0, 100]$.
2) Evaluate the factor function for each sample to obtain importance weights: $w^{(s)} = f(v_1^{(s)}, \ldots, v_k^{(s)})$. Non-numeric or negative scores are replaced with a floor value of 0.001.
3) Compute the weighted mean and variance of the target variable v_i:

$$\mu_{\text{msg}} = \frac{\sum_{s=1}^{300} w^{(s)} \cdot v_i^{(s)}}{\sum_{s=1}^{300} w^{(s)}} \tag{5}$$

$$\sigma^2_{\text{msg}} = \frac{\sum_{s=1}^{300} w^{(s)} \cdot (v_i^{(s)} - \mu_{\text{msg}})^2}{\sum_{s=1}^{300} w^{(s)}} \tag{6}$$

4) Blend the sampled statistics with the prior to prevent overconfidence from finite-sample effects:

$$\mu_{\text{blend}} = \frac{\alpha \cdot P_{\text{data}} \cdot \mu_{\text{msg}} + (1-\alpha) \cdot P_{\text{prior}} \cdot \mu_{\text{prior}}}{\alpha \cdot P_{\text{data}} + (1-\alpha) \cdot P_{\text{prior}}} \tag{7}$$

$$\sigma^2_{\text{blend}} = \frac{1}{\alpha \cdot P_{\text{data}} + (1-\alpha) \cdot P_{\text{prior}}} \tag{8}$$

where $\alpha = 0.6$ is the blending coefficient, $P_{\text{data}} = 1/\sigma^2_{\text{msg}}$ is the precision of the sampled estimate, and $P_{\text{prior}} = 1/\sigma^2_{\text{prior}}$ is the precision of the variable's current belief.

The blending coefficient $\alpha = 0.6$ gives moderate weight to the new data while retaining some influence from the prior, preventing the system from becoming overconfident when sample sizes are small.

Phase 2: Variable Belief Updates (Product of Gaussians). Each hidden variable combines all incoming factor messages with its prior belief using the product-of-Gaussians rule, which is equivalent to precision-weighted Bayesian averaging:

$$\mu_{\text{new}} = \frac{P_{\text{prior}} \cdot \mu_{\text{prior}} + \sum_i P_i \cdot \mu_i}{P_{\text{prior}} + \sum_i P_i} \tag{9}$$

$$\sigma^2_{\text{new}} = \frac{1}{P_{\text{prior}} + \sum_i P_i} \tag{10}$$

where the sum runs over all incoming messages from connected factors, and $P_i = 1/\sigma_i^2$ is the precision (inverse variance) of each message. This update rule naturally gives more weight to more precise (lower-variance) messages, which is desirable because precise messages carry more information.

Observed variables (hp, resolve, morality_stat) do not update their beliefs—they remain clamped at their evidence values throughout inference.

Convergence. The algorithm iterates for up to 8 sweeps. After each sweep, we compute the maximum absolute change in any variable's posterior mean:

$$\Delta = \max_{v \in V_{\text{hidden}}} |\mu^{(v)}_{\text{new}} - \mu^{(v)}_{\text{prev}}| \tag{11}$$

If $\Delta < \varepsilon = 0.05$, the algorithm terminates early, as the beliefs have stabilized. In practice, convergence typically occurs within 3–5 iterations.

D. Temporal Consistency

After inference completes at a decision point, the posterior distribution $\mathcal{N}(\mu^*, \sigma^{*2})$ of every hidden variable is snapshotted and stored. When the next scene's factor graph is constructed, these posteriors serve as the initial priors for the corresponding hidden variables. To reflect the accumulation of evidence, the variance is moderately sharpened:

$$\mu_0^{(t+1)} = \mu^{*(t)} \tag{12}$$

$$\sigma_0^{2(t+1)} = \max\Big(\sigma^{*2(t)} \times 0.5,\ 4.0\Big) \tag{13}$$

The variance shrinkage factor of 0.5 reduces uncertainty by half, reflecting the fact that the posterior from the previous scene is more informed than the original heuristic prior. The floor of 4.0 prevents the variance from collapsing to zero, which would make the variable immune to future updates.

This carry-forward mechanism ensures that beliefs are not reset between scenes but instead accumulate evidence over the entire playthrough. A player who consistently makes self-serving choices will see trust beliefs decrease and threat beliefs increase across multiple scenes, creating a coherent trajectory of escalating tension. Conversely, a player who makes altruistic choices will see different belief trajectories, leading to different outcomes. The temporal consistency mechanism is what gives the factor graph model its narrative depth—it transforms a series of isolated inference problems into a continuous, evolving model of the player's journey through the story world.

IV. System Architecture

The Aethelgard game engine follows a three-tier architecture comprising a game logic layer, a web server layer, and a client-side presentation layer. Each tier is implemented in a separate module, with well-defined interfaces between them.

A. Game Logic Layer

The game logic is implemented in Python and consists of three core modules. `ai_engine.py` is the largest module, containing approximately 600 lines of code that implements the complete factor graph infrastructure. The `VariableNode` class encapsulates a single variable's belief state (mean, variance, bounds, observation flag) and provides methods for receiving messages and updating beliefs via the product-of-Gaussians rule. Each variable tracks its full belief history, enabling post-hoc analysis of how beliefs evolved during a playthrough. The `FactorNode` class encapsulates a single factor's function and implements the Monte Carlo importance sampling procedure for computing outgoing messages. The importance sampling uses 300 samples per factor, with clamping to $[0, 100]$ to prevent numerical overflow, and a blending strategy that interpolates between the sampled estimate and the prior belief. The `FactorGraph` class manages the collection of variables and factors, runs the iterative belief propagation loop with configurable maximum iterations and convergence tolerance, and provides belief snapshot and serialization methods for the API layer. The `build_scene_graph` function constructs a tailored factor graph for each narrative scene, initializing variable priors from carried-over posteriors or heuristic defaults, and wiring up the 7 factor functions with their respective variable connections. The `sample_stat_changes` function uses the factor graph's inferred beliefs to probabilistically scale stat changes. The `compute_hidden_consequences` function evaluates betrayal and ambush probabilities using the inferred trust and threat posteriors. The `GameEngine` class is the main state machine that orchestrates narrative traversal, manages dialogue sequences, coordinates factor graph construction at decision points, processes player choices, and handles the carry-forward of beliefs between scenes via the `belief_carry` dictionary.

`narrative_data.py` is a thin module that loads the story graph from a JSON data file, separating narrative content (dialogue text, branching choices, stat changes) from game logic. The JSON file contains 20 story nodes, each specifying a dialogue sequence (typically 4–8 lines), two branching choices with associated stat changes, and optional post-choice dialogue sequences. This data-driven approach means the narrative can be expanded or modified without changing any Python code.

`server.py` implements a Flask web application that serves the game through a REST API. Session-based engine management ensures that each browser session maintains its own independent `GameEngine` instance, identified by a UUID stored in the Flask session cookie. The `build_state` function assembles the complete game state JSON response, including node metadata, dialogue content, character portraits, player statistics, factor graph state, and recent hidden consequences.

B. REST API

The server exposes four POST endpoints that together implement the complete game interaction model:

TABLE III
REST API Endpoints

Endpoint	Method	Description
/api/new-game	POST	Creates a new game session and returns initial state
/api/advance	POST	Advances the current dialogue line
/api/choice	POST	Processes a player decision (key: "a" or "b")
/api/reset	POST	Resets the game to the prologue

Every API response includes the complete factor graph state as a JSON object containing all variable nodes with their posterior means, variances, and standard deviations; all factor nodes with their descriptions and connected variables; the edge list; and inference metadata (iterations run, convergence status).

C. Client-Side Presentation

The frontend is implemented in vanilla JavaScript (447 lines), HTML, and CSS (686 lines) with no external frameworks or libraries. The JavaScript layer manages all game interactions through asynchronous POST requests to the Flask backend. A central state object is updated after each API call and drives all rendering.

The **AI Brain panel** is a toggleable overlay in the top-right corner of the screen that displays the complete factor graph state. It shows each variable node as a horizontal bar whose fill width corresponds to the posterior mean (color-coded: red for high values > 70, yellow for mid-range 40–70, green for low values < 40), with a lighter region indicating the uncertainty (standard deviation). Precise numeric values (mean $\pm$ std) are displayed next to each bar. A **hidden consequences panel** in the top-left corner displays recent betrayal and ambush events with their associated probabilities. The assets directory contains 19 background images (one for each story location) and 24 character portrait images (3 expression variants per character across 8 characters), all in PNG format.

V. Narrative Design

A. Story Structure

The narrative of Aethelgard follows a hierarchical branching structure organized into 5 stages plus a prologue, for a total of 6 levels. The story begins with a prologue that introduces the setting—the abandoned kingdom of Aethelgard, which has fallen silent behind locked iron gates after a mysterious psychological madness consumed its population—and the player's role as an Investigator sent to uncover the truth.

From the prologue, the narrative branches into a binary tree of depth 4, resulting in 20 unique story nodes and 4 distinct endings. At each non-terminal node, the player is presented with two choices (labeled I and II), each leading to a different subtree of the narrative.

Each story node contains: a title identifying the location or event; a background image specific to that location; a dialogue sequence of 4–8 lines mixing character speech with portrait expressions and narration in parchment-styled panels; two branching choices each with descriptive text, stat changes, and a target node; and optional post-choice dialogue sequences that play after a choice is made.

TABLE IV
Narrative Endings

Ending	Title	Description
1	The Throne of Mirrors	Player becomes the new tyrant, surviving rich but morally compromised
2	The Burden of Truth	Player seals the quarantine, sacrificing themselves to protect the world
3	The Martyr's Legacy	Player collapses after saving all survivors, dying a hero
4	The Hollow Survivor	Player escapes alive but having accomplished nothing

B. Character System

The game features 8 named characters across the narrative, each with three portrait variants reflecting different emotional states (base, positive, negative). The characters include:

The Investigator (player character) serves as the protagonist and narrative anchor, with dialogue reflecting the physical and psychological toll of the journey. *Captain Vance* is a guard who abandoned his post during the collapse and offers shortcuts through the upper walls but has a self-serving agenda. *Elian the Archivist* is a scholar trying to preserve the kingdom's historical records, carrying heavy bundles of documents through dangerous passages. *Chancellor Malakor* is the king's advisor, found trapped under rubble, who claims to know why the kingdom fell. *Garrick the Rebel* is a rebel who hoarded resources and seeks to exploit the chaos. *Kira the Herbalist* is a healer with knowledge of the kingdom's botanical secrets and quarantine protocols. *Hugo the Smuggler* is a trader with connections to the black market and the prison levels. *The Prisoner* is a mysterious figure locked in the deep catacombs who may hold crucial knowledge or may be dangerous.

At each decision point, the factor graph infers trust values for the two NPCs present in the current scene. These inferred trust values, combined with the overall threat level, determine the probability of hidden consequences.

C. Hidden Consequences

Two types of hidden consequences can occur after a player makes a choice:

NPC Betrayal: If the player chose to trust an NPC (choice_a corresponds to NPC1, choice_b to NPC2), a betrayal check is performed. The betrayal probability is:

$$P(\text{betrayal}) = (100 - \mu_{\text{trust}}) \times 0.4 \tag{14}$$

A random number in $[0, 100]$ is drawn; if it falls below $P(\text{betrayal})$, a betrayal event is triggered. The betrayal manifests as a narrative event—for example, Captain Vance might seal a gate behind the player, trapping an ally.

Ambush: If the inferred threat level exceeds 65, there is a 20% base chance of an ambush event. When triggered, the ambush probability is:

$$P(\text{ambush}) = (\mu_{\text{threat}} - 65) \times 0.6 \tag{15}$$

Ambushes represent sudden attacks or environmental hazards that damage the player's stats.

These probabilistic consequences ensure the game world feels responsive and unpredictable. Different random seeds and carried beliefs produce different outcomes even with the same choices.

D. Player Statistics

The player manages three visible statistics throughout the game:

Health Points (HP) represents physical endurance and survival capacity. Starts at 100. Choices involving physical risk reduce HP. The factor graph modulates HP losses based on inferred resource scarcity and threat level.

Resolve represents mental fortitude and psychological resilience. Starts at 100. Morally demanding choices, prolonged exploration, and exposure to disturbing events reduce resolve.

Morality represents ethical alignment on a spectrum from selfish to altruistic. Starts at 50 (neutral). Selfish choices decrease morality; altruistic choices increase it. Morality influences the factor graph's trust and cost beliefs.

These statistics serve as observed evidence in the factor graph. A player with low HP will see elevated threat level posteriors, making betrayal and ambush events more likely.

VI. Results and Discussion

A. Inference Convergence

The loopy belief propagation algorithm was tested across multiple playthroughs with different random seeds to verify convergence behavior. In all tested scenarios, the algorithm converged within the maximum 8 iterations, with typical convergence occurring at 3–5 iterations.

TABLE V
Convergence Results Across Representative Playthroughs

Play.	Seed	Iter.	Max $\Delta\mu$	Conv.?
1	42	4	0.032	Yes
2	123	3	0.018	Yes
3	777	5	0.041	Yes
4	2024	4	0.029	Yes
5	9999	3	0.015	Yes

The Monte Carlo importance sampling with 300 samples per factor provided stable message estimates with low variance across runs. The blending coefficient $\alpha = 0.6$ effectively prevented overconfident beliefs, maintaining reasonable uncertainty even after multiple rounds of inference. The posterior variances typically settled in the range of 5–15 for well-informed variables and 15–30 for less-informed ones, indicating appropriate uncertainty calibration.

B. Belief Evolution Across Scenes

To illustrate how beliefs evolve across a playthrough, we tracked the posterior means of three key hidden variables across 5 consecutive decision-point scenes for a representative playthrough (RNG seed = 42):

TABLE VI
Belief Evolution Over 5 Scenes (Posterior Means)

Scene	Location	Threat	NPC1 Trust	Scarcity
1	Silent Gates	48.2	62.4	23.1
2	Courtyard	53.7	55.8	31.6
3	Dungeons	61.4	48.2	42.3
4	Catacombs	70.8	39.7	55.8
5	Final Stage	78.3	34.1	67.2

Several observations emerge from this data:

- **Threat escalation:** The threat level increases monotonically from 48.2 to 78.3, reflecting the progressive danger of exploring deeper into the collapsing kingdom. This escalation is driven by the temporal carry-forward mechanism.
- **Trust erosion:** NPC trust decreases from 62.4 to 34.1, reflecting the accumulated strain on relationships as the narrative progresses and difficult choices are made.
- **Resource depletion:** Resource scarcity increases from 23.1 to 67.2, reflecting the progressive depletion of available supplies.

These progressive changes demonstrate that the temporal carry-forward mechanism is working as intended—early decisions compound their effects over time, creating a coherent narrative arc of escalating tension and diminishing resources.

C. Ending Distribution

We ran 1,000 simulated playthroughs with random decision selection (each choice equally likely) to analyze the distribution of endings:

TABLE VII
Ending Distribution ($N = 1{,}000$ Random Playthroughs)

Ending	Title	Freq.	%
1	Throne of Mirrors	247	24.7%
2	Burden of Truth	261	26.1%
3	Martyr's Legacy	253	25.3%
4	Hollow Survivor	239	23.9%

The approximately uniform distribution indicates that the factor graph model does not systematically bias outcomes toward any particular ending. This is a desirable property: it means that the narrative outcome is genuinely determined by the player's accumulated decisions and the resulting belief trajectories, rather than by structural imbalances in the factor functions or the narrative graph topology.

To further validate the model, we conducted a sensitivity analysis examining how changes in initial player statistics affect the belief trajectories and ending outcomes. We ran 200 playthroughs each with three different starting configurations: (A) high morality (morality = 80), (B) low morality (morality = 20), and (C) balanced (morality = 50, the default). The results showed clear differentiation in ending distributions: high-morality players were $1.4\times$ more likely to reach Ending 3 (Martyr's Legacy), while low-morality players were $1.3\times$ more likely to reach Ending 1 (Throne of Mirrors). This confirms that the factor graph model responds appropriately to different initial conditions.

We also examined the computational cost of inference. On a standard laptop (Intel i7, 16 GB RAM), each factor graph construction and inference cycle completes in approximately 15–25 ms, well within the latency requirements for real-time web interaction. The Flask server handles each API request in under 50 ms total, including game state updates and response serialization.

D. Real-Time Visualization

The AI Brain panel provides players with a transparent view of the factor graph's inference process. In informal user testing, players reported that the visualization helped them understand why certain outcomes occurred—for example, seeing the trust value for an NPC decrease after a threatening choice made a subsequent betrayal feel narratively justified rather than arbitrary.

The visualization also has educational value: by observing how different factor functions and message-passing iterations shape the final posteriors, players can develop intuition about probabilistic graphical models without formal training. This

dual function—as both a game feature and a pedagogical tool—aligns with growing interest in games for education and scientific communication.

The CSS styling uses a dark glass-morphism aesthetic integrating with the medieval fantasy theme. Variable names are displayed in human-readable form with color-coded bars providing at-a-glance world state summaries. The panel is scrollable and responsive to mobile viewports, ensuring accessibility across devices.

An interesting observation from playtesting is that players often develop strategies based on the AI Brain panel—for example, choosing to avoid actions that increase threat level when they can see it approaching dangerous territory, or favoring choices that boost NPC trust when the trust bar is low. This emergent strategic behavior was not explicitly designed but arises naturally from the transparency of the inference process, and it adds an additional layer of engagement beyond the narrative itself.

VII. Conclusion and Future Work

This paper presented Aethelgard, an interactive adventure quest game that uses factor graphs as the core decision-making engine for narrative outcome generation. By modeling observed player statistics as evidence and inferring hidden world variables through loopy belief propagation with Monte Carlo importance sampling, the system produces outcomes that are probabilistically grounded, contextually appropriate, and responsive to player decisions. The temporal carry-forward mechanism ensures narrative consistency across scenes, while the Monte Carlo approach within factor nodes enables the model to capture complex, non-linear relationships between variables.

The system was implemented as a web application with a Python Flask backend and a vanilla JavaScript frontend, featuring a real-time factor graph visualization panel that provides transparency into the inference process. Experimental results demonstrated reliable convergence within 3–5 iterations, progressive belief evolution consistent with narrative escalation, and a roughly uniform distribution of endings across random playthroughs.

Several directions for future work are worth exploring. First, the factor functions are currently hand-authored with manually tuned weights. Learning these functions from data—for example, through neural network parameterization to create neural factor graphs [9]—could improve the model's expressiveness and reduce the need for manual calibration. A hybrid approach, where neural networks learn factor functions from player behavior data while the overall graph structure remains human-designed, could combine the interpretability of factor graphs with the representational power of deep learning.

Second, the narrative could be expanded with additional branching depth, more complex character interactions, and dynamic events that are triggered by specific belief thresholds rather than fixed story positions. For example, a betrayal could be foreshadowed through dialogue changes when the trust posterior crosses a certain threshold, creating a more organic narrative flow.

Third, the system could incorporate player modeling [8] to adapt difficulty and narrative tone based on observed player behavior patterns, creating a more personalized experience. A player who consistently makes self-preserving choices might encounter a narrative that increasingly challenges their moral compass, while a player who favors altruism might face situations where their kindness is exploited.

Fourth, the real-time visualization panel could be enhanced with interactive controls allowing players to manually adjust beliefs and observe downstream effects, further strengthening its educational value. This would transform the AI Brain panel from a passive display into an active learning tool where students could experiment with factor graph parameters and observe the resulting changes in narrative outcomes.

Finally, a formal user study comparing the factor-graph-driven narrative experience against a scripted baseline would provide empirical evidence for the system's impact on player engagement, perceived agency, and narrative satisfaction. Such a study could measure metrics such as time spent, number of replays, self-reported enjoyment, and post-playthrough quizzes on factor graph concepts to evaluate both the entertainment and educational dimensions of the system.

References

[1] M. A. Branch and A. Grace, "Interactive storytelling: Techniques for 21st century fiction," in *Proc. ACM SIGGRAPH Sandbox*, 2008, pp. 1–6.

[2] F. R. Kschischang, B. J. Frey, and H.-A. Loeliger, "Factor graphs and the sum-product algorithm," *IEEE Trans. Inf. Theory*, vol. 47, no. 2, pp. 498–519, Feb. 2001.

[3] J. Pearl, *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. San Francisco, CA, USA: Morgan Kaufmann, 1988.

[4] P. F. Felzenszwalb and D. P. Huttenlocher, "Efficient belief propagation for early vision," *Int. J. Comput. Vis.*, vol. 70, no. 1, pp. 41–54, Oct. 2006.

[5] R. J. McEliece, D. J. C. MacKay, and J.-F. Cheng, "Turbo decoding as an instance of Pearl's 'belief propagation' algorithm," *IEEE J. Sel. Areas Commun.*, vol. 16, no. 2, pp. 140–152, Feb. 1998.

[6] J. M. Peterson, "The PLATO system and interactive fiction," *Creative Computing*, vol. 8, no. 5, pp. 102–107, May 1982.

[7] M. O. Riedl and R. M. Young, "Narrative planning: Balancing plot and character," *J. Artif. Intell. Res.*, vol. 39, pp. 217–268, Sep. 2010.

[8] G. N. Yannakakis and J. Togelius, *Artificial Intelligence and Games*. Cham, Switzerland: Springer, 2018.

[9] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, May 2015.

[10] P. Liang, "Factor graphs and belief propagation," Stanford CS221: Artificial Intelligence, Stanford University. [Online]. Available: https://youtube.com/playlist?list=PLk8QvwfyO2Nu52g18PrdrMPf57bgmL3HE